

SpiceRAT Malware Analysis

Reverse Engineering and Detection Engineering Report

Prepared by

Ashvin Acharya

SOC Analyst | Malware Analysis | Detection Engineering

Connect

LinkedIn

<https://www.linkedin.com/in/ashvin-aacharya-a08553308>

GitHub

<https://github.com/0xAshvin/Soc-investigation>

X (Twitter)

<https://x.com/0xAshvin>

Platform

Let'sDefend

Challenge

SpiceRAT (Medium)

Tools Used

- IDA Free
 - Ghidra
 - HashCalc
-

Date

July 2026

1. Executive Summary

Executive Summary

SpiceRAT is a Windows-based Remote Access Trojan (RAT) designed to establish remote communication with a command-and-control (C2) server and execute malicious payloads directly in memory. During the analysis, the malware was observed performing host reconnaissance, resolving Windows APIs dynamically, creating a mutex to prevent multiple instances, enumerating running processes, and downloading additional payloads from its C2 infrastructure.

Unlike traditional malware that relies on files written to disk, SpiceRAT employs the **RunPE** technique to execute payloads in memory, reducing its forensic footprint and making detection more challenging. The malware also dynamically resolves networking APIs at runtime, allowing it to avoid static import-based detection and improve its resistance against security products.

The objective of this analysis was to reverse engineer the malware, identify its functionality, answer the investigation questions, extract Indicators of Compromise (IOCs), map observed behaviors to the MITRE ATT&CK framework, and develop detection rules that can assist defenders in identifying similar threats within enterprise environments.

2. Lab Overview

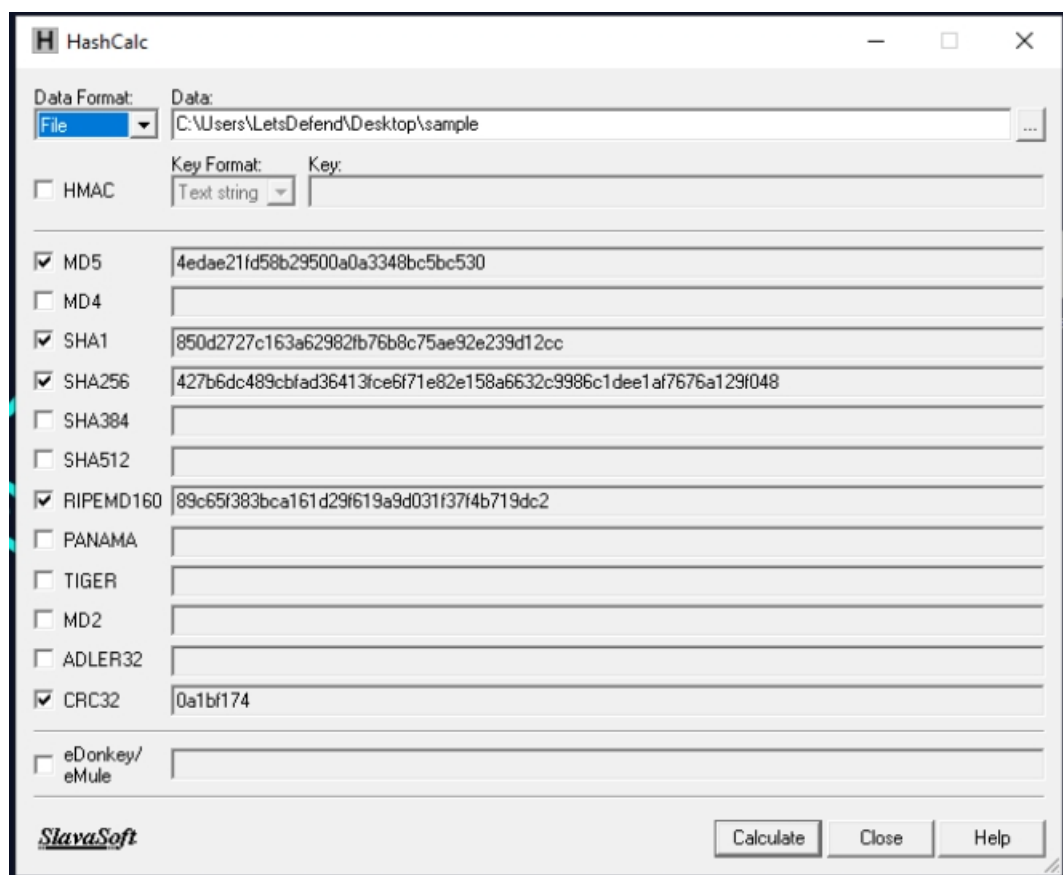
The objective of this challenge was to reverse engineer a **SpiceRAT** malware sample and answer a series of analysis questions based on its behavior. The investigation focused on identifying key artifacts such as the file hash, mutex, process enumeration, dynamic API resolution, payload execution technique, and exported functions.

The analysis was performed primarily using **Ghidra**, while **IDA Free** was used only for verifying the exported function in the final question. Since Ghidra labels unidentified functions as `FUN_XXXXXXXX` and IDA uses `sub_XXXXXXXX`, the equivalent IDA function names were used where required to match the Let'sDefend answer format.

The following sections explain how each answer was identified, along with the reasoning behind the analysis.

3. Static Analysis

The analysis began with basic static triage to understand the sample before diving into reverse engineering. The first step was to calculate the file's **MD5 hash** using **HashCalc**. Generating the hash provides a unique identifier for the malware and allows analysts to correlate the sample with threat intelligence platforms such as VirusTotal.

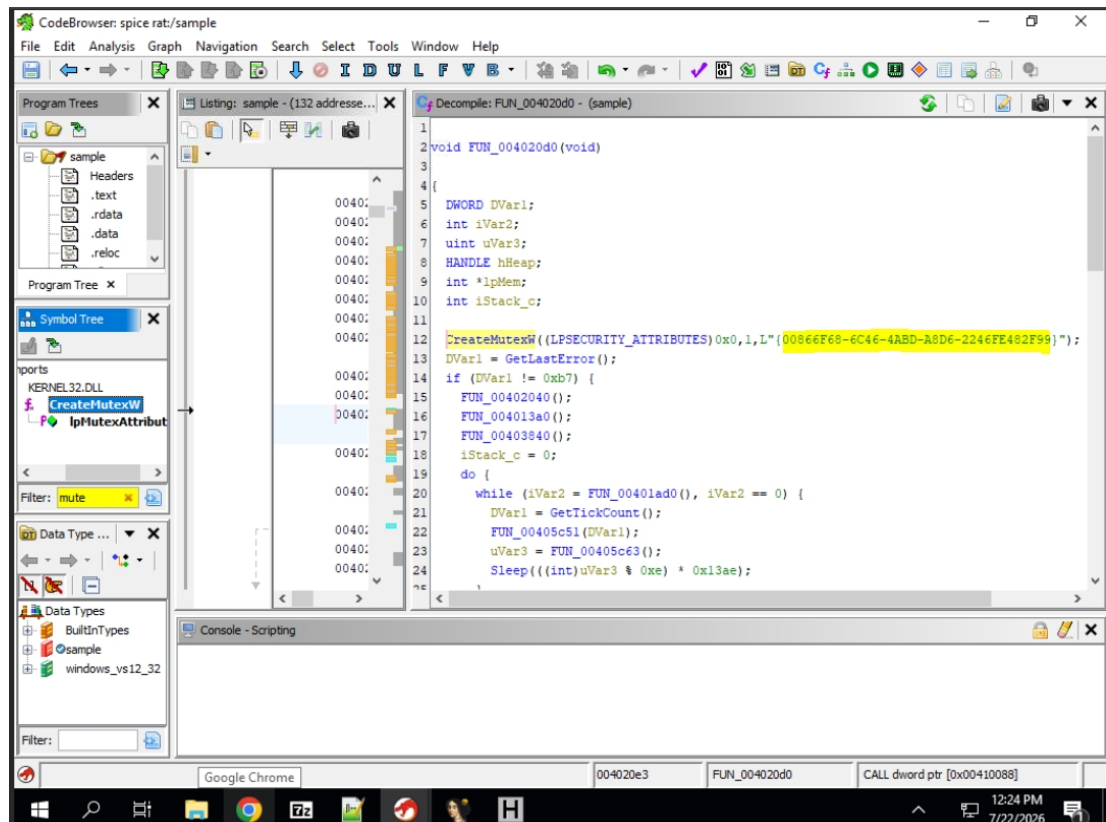


 **Figure 2 – Calculating the MD5 hash using HashCalc**

Following the hash verification, the binary was loaded into **Ghidra** for code analysis. A quick review of the imported APIs and string references provided an overview of the malware's functionality and highlighted areas that required further investigation.

4. Initial Code Analysis

One of the first APIs that drew attention was `CreateMutexW`, as malware commonly uses mutexes to prevent multiple instances from executing on the same system. By following the cross-reference in Ghidra, the function responsible for mutex creation was identified. The mutex name was hardcoded as a GUID, confirming that the malware checks whether another instance is already running before continuing execution.



 **Figure 3 – Mutex creation using `CreateMutexW`**

During the same stage of analysis, references to `GetUserNameW` were examined to understand what host information the malware collected. The API retrieves the username of the currently logged-in user. The initialized buffer contained the string "ROLEX", which served as the expected value during the challenge.

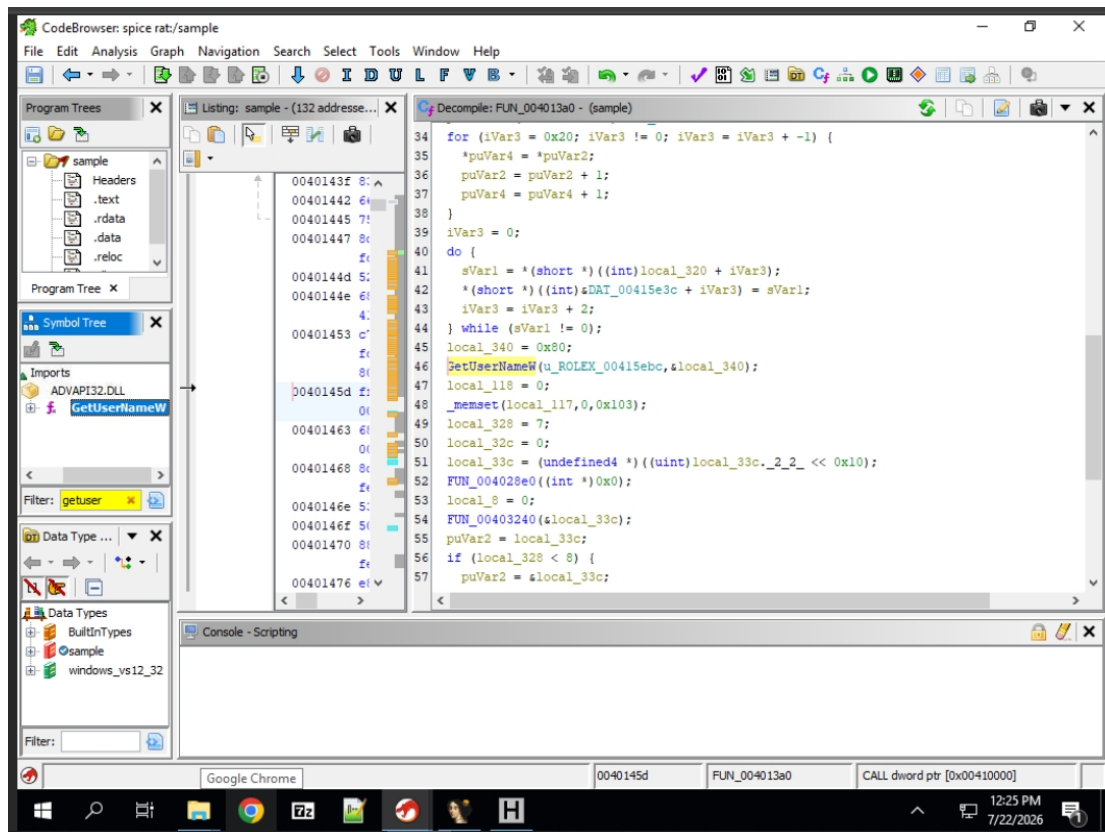


Figure 4 – Username retrieval using GetUserNameW

The process enumeration routine was then identified by locating calls to CreateToolhelp32Snapshot, Process32FirstW, and Process32NextW. These APIs confirmed that the malware enumerates running processes before proceeding with its remaining functionality. Tracing this routine also helped identify the corresponding function referenced later in the analysis.

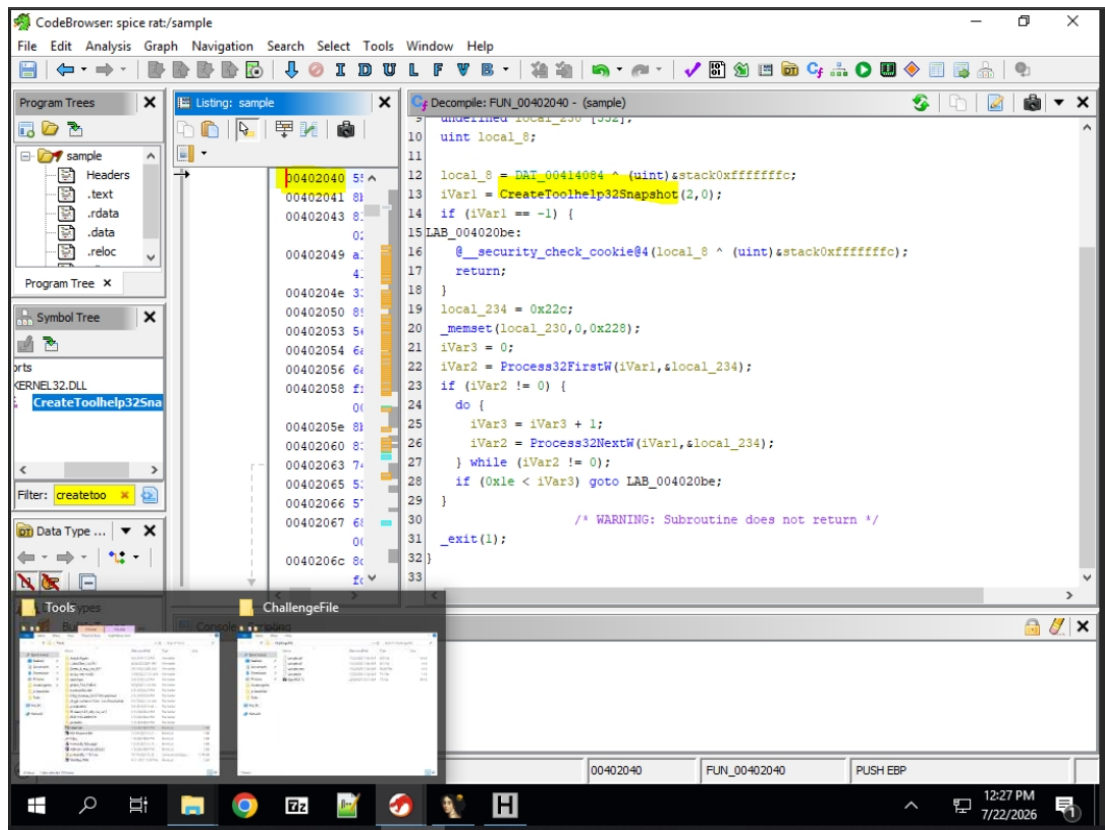


Figure 5 – Process enumeration routine

5. Reverse Engineering Analysis

After understanding the malware's initial behavior, the next objective was to identify how it performed network communication and executed additional payloads. During the code review, it became evident that the malware did not rely entirely on its import table. Instead, several Windows APIs were resolved dynamically at runtime, a common technique used to hinder static analysis.

To locate the resolver function, string references and API calls were traced in **Ghidra** until the routine responsible for loading Windows APIs was identified. This function resolved multiple networking-related APIs, including `HttpSendRequestW`, confirming that the malware establishes communication with its command-and-control (C2) server through the WinINet library.

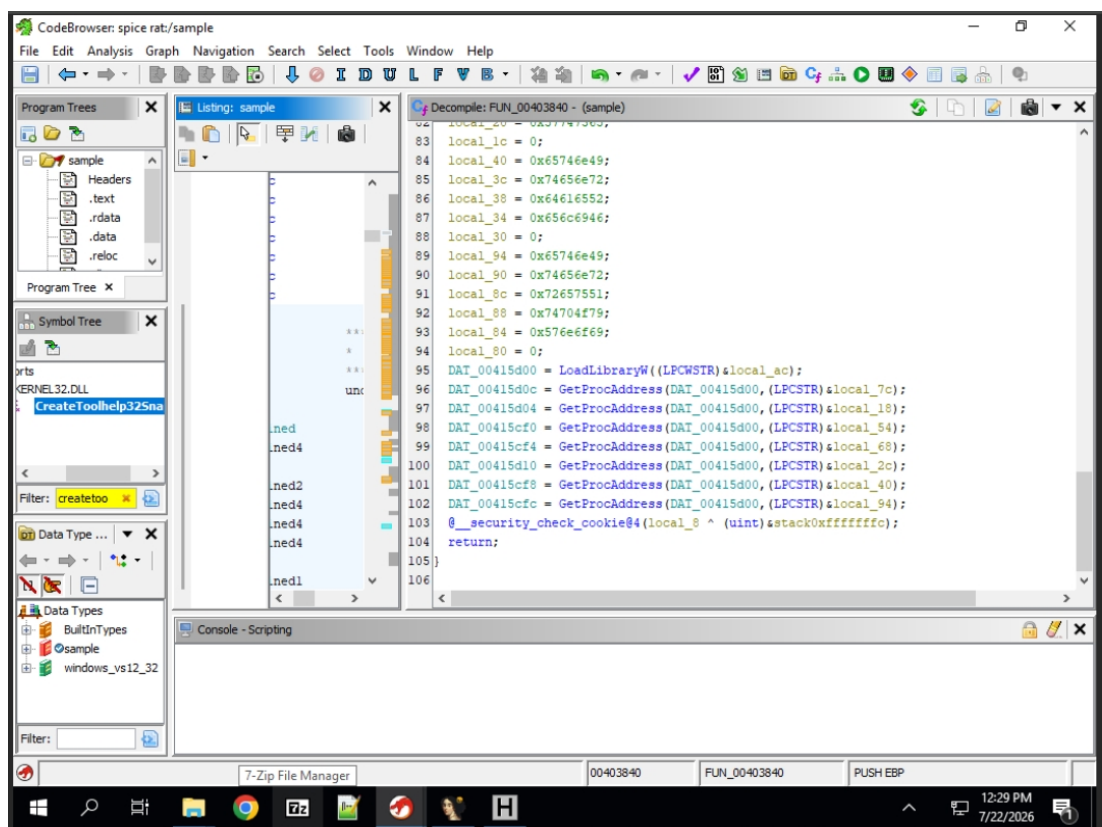


Figure 6 – Dynamic API resolution routine



Figure 7 – Resolution of WinINet APIs

Note: Ghidra automatically labels unidentified functions using the FUN_ convention, whereas IDA represents the same functions as sub_ convention. Since the challenge expected the IDA naming format, the corresponding sub function names were used where required during submission.

Further analysis of the execution flow revealed that the malware receives additional data from the remote server and loads it directly into memory. Instead of dropping a file onto disk, the loader allocates memory, copies the Portable Executable (PE), resolves imports, applies relocations, updates memory permissions, and finally transfers execution to the loaded image. This workflow matches the **RunPE** technique, allowing the malware to execute payloads while leaving minimal artifacts on the filesystem.

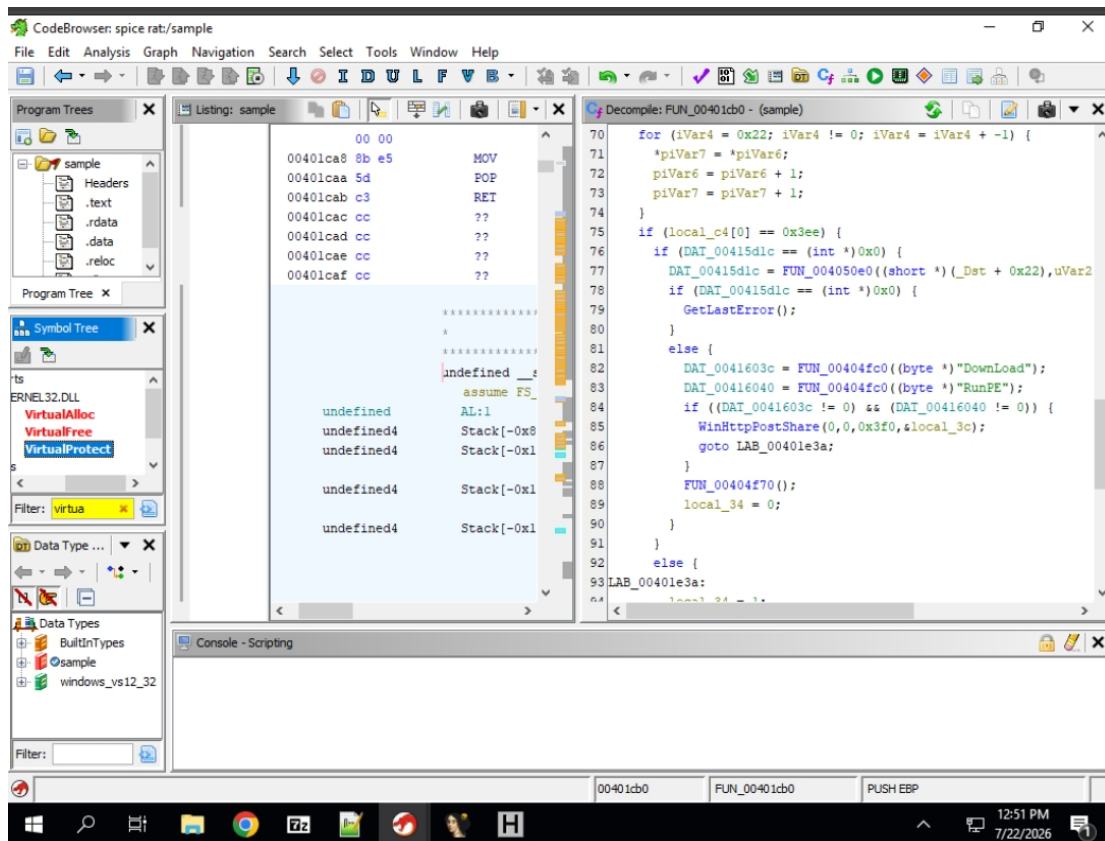
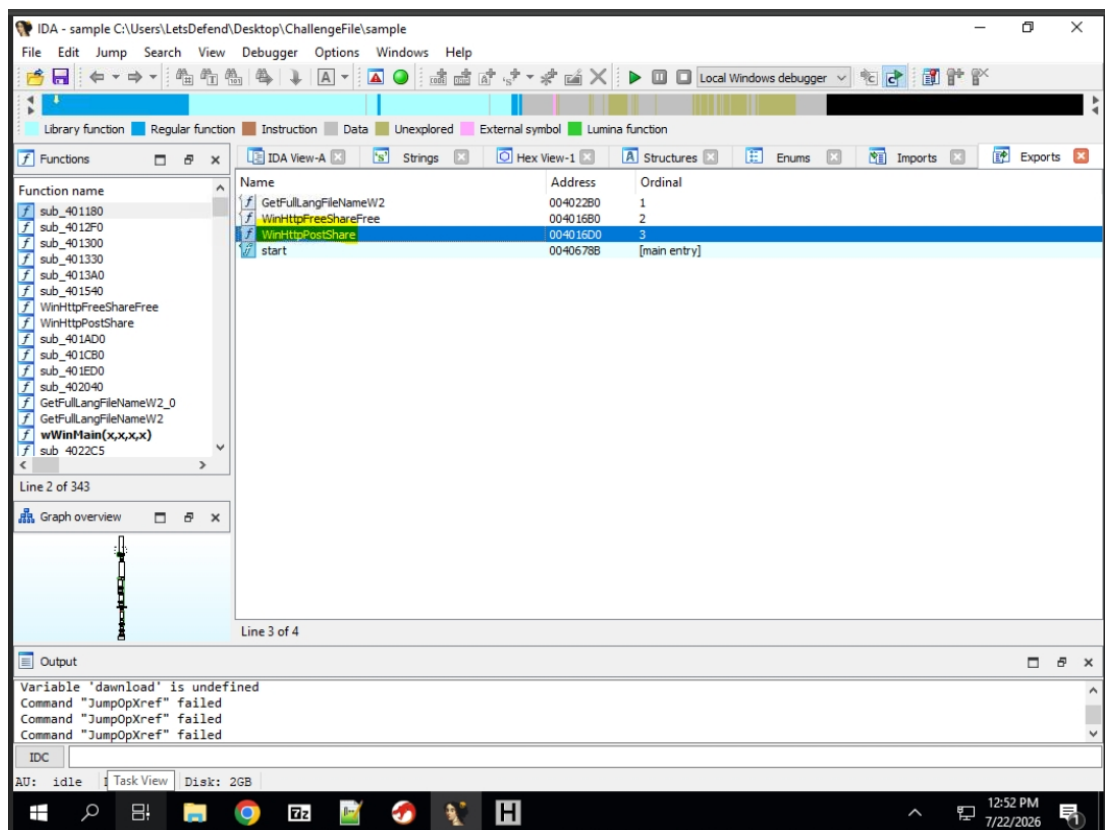


Figure 8 – RunPE implementation

The final stage of the investigation focused on identifying the exported function responsible for handling network communication. Although the analysis was primarily performed in **Ghidra**, **IDA Free** was used for this step because it provides a clearer view of the export table. Reviewing the exported functions confirmed that **WinHttpPostShare** is responsible for receiving data from the C2 server, completing



the reverse engineering process.

Figure 9 – Export table showing WinHttpPostShare

6. Malware Behavior & Findings

The analysis revealed that **SpiceRAT** is designed to collect basic host information, communicate with a remote command-and-control (C2) server, and execute additional payloads directly in memory. Throughout the reverse engineering process, the malware demonstrated several techniques that are commonly used to evade detection and complicate static analysis.

Instead of relying on a complete import table, the malware dynamically resolves critical Windows APIs at runtime. This technique reduces the number of visible imports and makes it more difficult to understand the malware's capabilities through static inspection alone.

To maintain stable execution, the malware creates a unique mutex, ensuring that only one instance runs on the compromised system. It also gathers basic information such as the current username and enumerates running processes before proceeding with the remaining stages of execution.

Communication with the C2 server is performed through dynamically resolved WinINet APIs. Once a response is received, the malware loads the downloaded payload directly into memory using the **RunPE** technique. By avoiding writing the payload to disk, the malware significantly reduces forensic artifacts and increases the likelihood of bypassing traditional file-based security solutions.

8. Indicators of Compromise (IOCs)

The malware sample was identified using its MD5 hash, which can be used to correlate the file across threat intelligence platforms such as VirusTotal and assist in future investigations.

IOC Type	Value
MD5 Hash	<4EDAE21FD58B29500A0A3348BC5BC530>

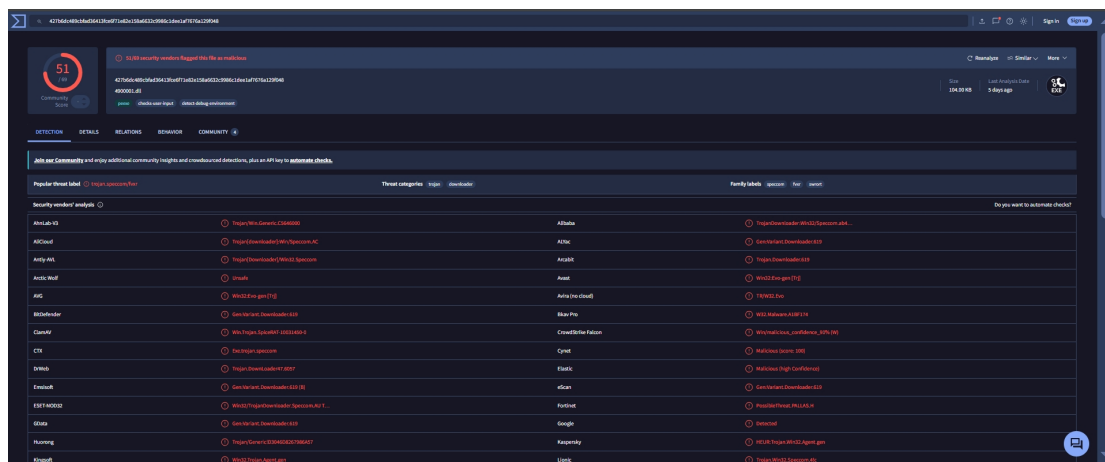


 Figure 10 – VirusTotal analysis of the malware sample

The VirusTotal analysis classified the sample as malicious, with multiple security vendors detecting it as a Trojan/Remote Access Trojan (RAT). This further validates the findings obtained during the reverse engineering process and confirms the malicious nature of the sample.

9. Conclusion

This investigation demonstrated a structured reverse engineering approach to analyzing the **SpiceRAT** malware sample. Starting with basic static analysis and progressing through code analysis in **Ghidra**, the malware's execution flow, dynamic API resolution, process enumeration, networking behavior, and in-memory payload execution were successfully identified.

Although Ghidra was sufficient for the majority of the investigation, **IDA Free** was used during the final stage to inspect the export table and verify the exported function responsible for network communication. The combination of both tools enabled accurate identification of the malware's behavior and all required challenge artifacts.

Overall, the sample demonstrates several techniques frequently observed in modern malware, including dynamic API resolution, process discovery, and RunPE-based in-memory execution, highlighting the importance of reverse engineering alongside traditional static analysis when investigating advanced threats.

Appendix - Challenge Answers

Artifact	Value
MD5 Hash	(4EDAE21FD58B29500A0A3348BC5BC530)
Mutex	{00866F68-6C46-4ABD-A8D6-2246FE482F99}
Process Enumeration Function	0x00402040
Username	ROLEX
Dynamic API Resolver	sub_403840
API at 0x4039E8	HttpSendRequestW
Payload Execution Technique	RunPE
Exported Function	WinHttpPostShare